MAD 1 WEEK 6

Lecture 1: L6.1 - API Design

Distributed Software Architecture

Distributed software architecture refers to a design approach where the components and functions of a software application are distributed across multiple computers connected over a network.

Web Architecture

Web architecture is the overall structure of a website or web application, including its design, implementation, and deployment. It involves the use of technologies and protocols such as HTML, CSS, JavaScript, and HTTP to build and deliver web pages and applications to users.

1. Client-Server Architecture

Client-server architecture is a software architecture where the client and the server are two separate entities that communicate with each other over a network. The client requests data from the server, and the server provides the requested data.

Example:

When you browse a website, your web browser (client) sends a request to the web server asking for a specific web page. The web server processes the request and sends back the requested web page, which your browser then displays.

a. Clear Separation of Concerns

- The client handles the user interface and interactions, while the server manages data processing, business logic, and storage.
- This separation allows for easier maintenance and scalability.

b. Distributed Nature

- The client and server can run on different devices connected over a network.
- Allows users to access resources and services from remote servers, making it suitable for large-scale deployments.

2. Stateless Architecture

Stateless architecture means the server does not maintain any state about the client.

Example:

When you search for different items on an e-commerce website, each search request is stateless. The server processes each search request independently without remembering your previous searches or interactions.

a. Stateless Communication

- Each client request to the server is independent and does not carry information about previous interactions.
- The server treats each request as a new one without relying on any previous context.

b. Simplified Server Management

- Stateless design simplifies server management as the server doesn't need to maintain session data for each client continuously.

c. Scalability

- Stateless systems are easier to scale horizontally, as any server can handle any client request without relying on shared state information.

3. Layered Architecture

Layered architecture divides an application into multiple layers, each responsible for a specific task, making the application easier to develop, maintain, and scale.

Example:

In a three-tier architecture:

- The front-end handles user interface interactions.
- The middle-tier processes business logic.
- The back-end stores data.
- Each layer communicates only with adjacent layers, promoting modularity.

a. Modular Design

- Web architecture often adopts a layered system, dividing the application into tiers (e.g., presentation, application logic, data storage).

b. Flexibility and Reusability

- Layers are loosely coupled, allowing changes in one layer without affecting others.
- Promotes flexibility and reusability of components.

c. Easier Maintenance

- Layered systems are easier to maintain and update since modifications can be confined to specific layers.
- Simplifies debugging and troubleshooting.

4. Cacheability

Cacheability refers to storing data in a cache so that it can be accessed more quickly later, improving web application performance by reducing server requests.

Example:

After loading a web page, your browser caches static resources (e.g., images, CSS files). When revisiting the page, these resources can be retrieved from the cache, reducing loading time.

a. Caching for Performance

- Cacheability allows web clients and intermediaries (e.g., proxies) to store server responses temporarily.
- Reduces server load and improves response times.

b. Reduced Latency

- Cached responses lead to reduced latency as clients can retrieve resources from the cache without making a new server request.

c. Cache Control Headers

- Web servers use cache control headers (e.g., Cache-Control, Expires) to instruct clients and intermediaries on caching behavior.
- Developers can control cache duration and content freshness.

5. Uniform Interface

A uniform interface is a set of constraints that define how resources should be represented and accessed in a RESTful web application. These constraints ensure consistency and ease of use.

Example:

A RESTful API provides a uniform interface for interacting with resources, using standard HTTP methods:

- **GET**: Retrieve a resource.
- **POST**: Create a new resource.
- **PUT**: Update a resource.
- **DELETE**: Remove a resource.

For instance:

- `GET /users` retrieves a list of users.
- `POST /users` creates a new user.

a. Consistent Interaction

- Provides a standardized way for clients to interact with resources, making the web easier to understand and use.

b. Resource-Based URLs

- Resources are identified by unique URLs, allowing clients to access and manipulate resources using standard HTTP methods.

Lecture 2: Standard HTTP Methods & REST

HTTP Methods

HTTP (HyperText Transfer Protocol) methods are used to perform operations on resources in web applications. The most commonly used HTTP methods are:

- **GET: Retrieves data from the server without modifying it. It is idempotent, meaning multiple identical requests produce the same result.**
- **POST: Sends data to the server to create a new resource. It is not idempotent, as multiple requests may create multiple resources.**
- **PUT: Updates an existing resource or creates a new one if it does not exist. It is idempotent.**
- **DELETE: Removes a resource. It is idempotent, as multiple delete operations on the same resource have the same effect.**

REST (REpresentational State Transfer)

REST is an architectural style for designing web services that rely on HTTP methods to manipulate resources.

Key Principles of RESTful Web Services

1. **Stateless: The server does not maintain client session information.**
2. **Cacheable: Responses can be cached to improve performance.**
3. **Client-Server Architecture: The client and server are separate, interacting only through requests and responses.**
4. **Uniform Interface: Standard HTTP methods (GET, POST, PUT, DELETE) are used for resource manipulation.**
5. **Resource Representation: Resources are represented in formats such as JSON, XML, YAML, or HTML.**
6. **Identification of Resources: Resources are uniquely identified by URIs (Uniform Resource Identifiers).**

Idempotent Operations

An operation is idempotent if applying it multiple times does not change the result beyond the first application.

- **GET: Always returns the same result without modifying the resource.**
- **PUT: Updates or creates a resource; multiple identical requests yield the same result.**
- **DELETE: Deleting an already deleted resource has no further effect.**

However, POST is not idempotent since sending multiple requests may create duplicate resources.

REST vs CRUD

Feature	REST (Web Services)	CRUD (Database Operations)
Definition	REST is an architectural style for designing APIs.	CRUD represents four fundamental operations on data.
Underlying Principles	Uses HTTP protocol and methods.	Protocol-independent; can be implemented in various ways.
Resource Identification	Identified by URIs.	Identified by names or IDs.

Representation	**Uses JSON, XML, YAML, or HTML.**	**Can be stored in any format, including relational databases.**
Operations	**Uses GET, POST, PUT, DELETE.**	**Uses CREATE, READ, UPDATE, DELETE.**

API Data Transfer Formats

Web services use different formats to transfer data between clients and servers. Common formats include:

1. HTML (HyperText Markup Language)

HTML is used for rendering web pages but is inefficient for data exchange.

Example:

```
<html>

<head><title>API Response</title></head>

<body>

<h1>List of Users</h1>

<ul>

  <li><strong>User 1</strong>

    <ul>

      <li>Name: John Doe</li>

      <li>Email: johndoe@example.com</li>

      <li>Phone: 123-456-7890</li>

    </ul>

  </li>

</ul>
```

```
</body>

</html>
```

2. JSON (JavaScript Object Notation)

A lightweight, human-readable format widely used for APIs.

Example:

```
[

  {

    "name": "John Doe",

    "email": "johndoe@example.com",

    "phone": "123-456-7890"

  },

  {

    "name": "Jane Doe",

    "email": "janedoe@example.com",

    "phone": "987-654-3210"

  }

]
```

3. XML (Extensible Markup Language)

More structured than JSON but less efficient.

Example:

```
<users>

    <user>
```

```
      <name>John Doe</name>

      <email>johndoe@example.com</email>

      <phone>123-456-7890</phone>

    </user>

  </users>
```

4. YAML (Yet Another Markup Language)

A human-readable format similar to JSON but less widely supported.

Example:

```
users:

  - name: John Doe

    email: johndoe@example.com

    phone: 123-456-7890

  - name: Jane Doe

    email: janedoe@example.com

    phone: 987-654-3210
```

L6.3: REST APIs - Example

This example demonstrates how to use the MediaWiki REST API to search Wikipedia.

1. Using cURL

cURL is a command-line tool for transferring data. Example request:

```
curl
"https://en.wikipedia.org/w/rest.php/v1/search/page?q=earth&limit=3"
```

This searches for "earth" in Wikipedia and returns up to 3 results.

2. Using Python (requests library)

```python
import requests


search_query = 'earth'

number_of_results = 3

url = 'https://en.wikipedia.org/w/rest.php/v1/search/page'

headers = {'User-Agent': 'MediaWiki REST API docs examples/0.1'}


response = requests.get(url, headers=headers, params={'q': search_query,
'limit': number_of_results})

data = response.json()

print(data)
```

3. Parameters

- `q`: **Search query (e.g., `"earth"`)**
- `limit`: **Number of results to return**

4. Responses

Status Code	Description
200	Success (returns search results)
200	No results found (empty response)
400	Missing or invalid query parameter
500	Internal server error

5. Example Response

```
{
  "pages": [
    {
      "id": 9228,
      "key": "Earth",
      "title": "Earth",
      "description": "Third planet from the Sun",
      "thumbnail": { "url": "//upload.wikimedia.org/...jpg" }
    },
    {
      "id": 2126501,
```

```
      "key": "Google_Earth",
      "title": "Google Earth",
      "description": "3D globe-based map program owned by Google",
      "thumbnail": { "url": "//upload.wikimedia.org/...svg" }
    },
    {
      "id": 19331,
      "key": "Moon",
      "title": "Moon",
      "description": "Natural satellite orbiting the Earth",
      "thumbnail": { "url": "//upload.wikimedia.org/...jpg" }
    }
  ]
}
```

L6.5: OpenAPI

OpenAPI (Swagger) is a specification for describing RESTful APIs.

1. Key Features

1. **API Description Language**
   - Uses YAML or JSON to define API structure.
   - Acts as a contract between API providers and consumers.
2. **Endpoint Documentation**

    - **Specifies HTTP methods (`GET`, `POST`, `PUT`, `DELETE`).**
    - **Includes request parameters, body format, and response schema.**
3. **Tooling and Interoperability**
    - **Supports code generation, validation, and interactive documentation.**
    - **Compatible with multiple programming languages.**
4. **OpenAPI Specification Components**
    - **API Version and Paths: Defines API version and endpoints.**
    - **HTTP Methods and Parameters: Lists supported methods and required parameters.**
    - **Request and Response Schemas: Specifies input/output data formats.**
    - **Authentication and Security: Details authentication methods.**
    - **Error Handling: Defines error codes and messages.**
    - **API Documentation: Acts as a reference for developers.**
    - **Data Models: Describes data structures used in the API.**